AI-Driven Automation of Finite Element Simulations using LangChain and Code_Aster

This project was carried out as a training internship for the students at Politecnico di Torino, in collaboration with S.T.A. DATA S.r.l.

Authors:
Saeideh Mohammadikish — s329781
Amir Yarmohamadi — s329783

University Promoter:
Politecnico di Torino

Academic Tutor:
Prof. Gianvito Urgese — gianvito.urgese@polito.it

Hosting Entity:
S.T.A. DATA S.r.l.

Company Tutor:
Ing. Davide Seni — davide.seni@stadata.com

November 2025

Table of Contents

1. Introduction ***2***

2. LangChain Overview ***3***

3. Code_Aster Overview ***4***

3.1. General Description 4

3.2. Syntax and Input Files 4

3.3. Execution and API Integration 5

3.4. Advantages over Commercial Alternatives 5

4. System Architecture ***5***

4.1. High-Level Workflow 6

4.2. Folder Structure and Responsibilities 6

4.3. Data and Execution Flow 7

5. Code Implementation ***7***

5.1. geo_generator.py — Geometry Creation 7

5.2. mesher_gmsh.py — Meshing Process 8

5.3. comm_generator.py — Solver Configuration 8

5.4. export_generator.py — Export Preparation 9

5.5. run_aster_tool.py — Simulation Execution 9

5.6. visualize_tool.py — Results Visualization 9

6. Results and Discussion ***10***

6.1. Example of Successful Runs 10

6.2. Interpretation of Generated Plots 11

6.3. Performance and Automation Improvements 11

6.4. Challenges and How They Were Solved 11

7. Conclusion and Future developments ***12***

7.1. Achievements 12

7.2. Key Learnings 13

7.3. Possible Extensions 13

7.4. Disclaimer 13

8. Appendix ***15***

8.1. Full Execution Log 15

8.2. Example Generated Files 18

1. Introduction

In recent years, the integration of Artificial Intelligence (AI) into engineering workflows has opened new opportunities for automating repetitive and knowledge-intensive tasks. One of the most promising applications lies in the field of Finite Element Analysis (FEA), where simulation setup, execution, and post-processing often require significant manual effort and expertise. This internship project aims to explore how Large Language Models (LLMs) can assist in automating and managing FEA processes through the use of the LangChain framework and the open-source solver Code_Aster.

Code_Aster, originally developed by EDF (Électricité de France), is a powerful and open-source finite element software for structural and thermal analysis. It supports a wide range of simulations including linear and nonlinear mechanics, dynamics, and thermomechanics. One of its main advantages is its flexibility: Code_Aster can be used both through a graphical interface (via Salome-Meca) and through command-line or API-based automation, making it ideal for integration with AI-driven systems. Combined with Gmsh for geometry creation and meshing, it provides a robust environment for end-to-end numerical analysis—from model generation to result visualization.

Compared to other commercial alternatives such as ANSYS, Abaqus, or COMSOL, Code_Aster offers full transparency, scriptability, and cost efficiency, which are crucial for research and automation workflows.

The LangChain framework acts as the intelligent layer of the system, enabling the LLM to reason, call specific tools, and manage complex interactions. By connecting LangChain agents to customized Python tools, the model can generate geometry files, prepare solver input decks, execute simulations, and visualize results autonomously. This approach demonstrates how AI can enhance existing FEA workflows by reducing human intervention and bridging the gap between natural language instructions and executable simulation commands.

The goal of this internship was to design and implement a prototype of an AI-assisted pipeline capable of running structural analyses in Code_Aster autonomously. The system was built to understand user queries, generate the corresponding input files, execute the solver, and visualize the outputs—all through a modular and explainable structure. This work represents a step toward the digital transformation of traditional FEA workflows, combining engineering knowledge with the reasoning capabilities of modern language models.

During the internship period, Amir Yarmohamadi and Saeideh Mohammadikish collaboratively developed and tested the entire pipeline, working on both the conceptual design and the technical implementation. Their contribution included defining the system architecture, implementing the LangChain-based tools, configuring the Code_Aster environment, and validating the numerical results through real simulations. The internship provided an opportunity to combine their backgrounds in structural engineering and artificial intelligence, demonstrating how interdisciplinary collaboration can lead to more intelligent and automated simulation workflows

To manage the project effectively during the 250 hour, approximately two-month, development period, a task-level Gantt chart was created. It highlights how the work evolved from early learning phases to system integration and documentation. This visual timeline, shown in Figure 1, illustrates the sequential and overlapping stages of the project and provides context for the development workflow.

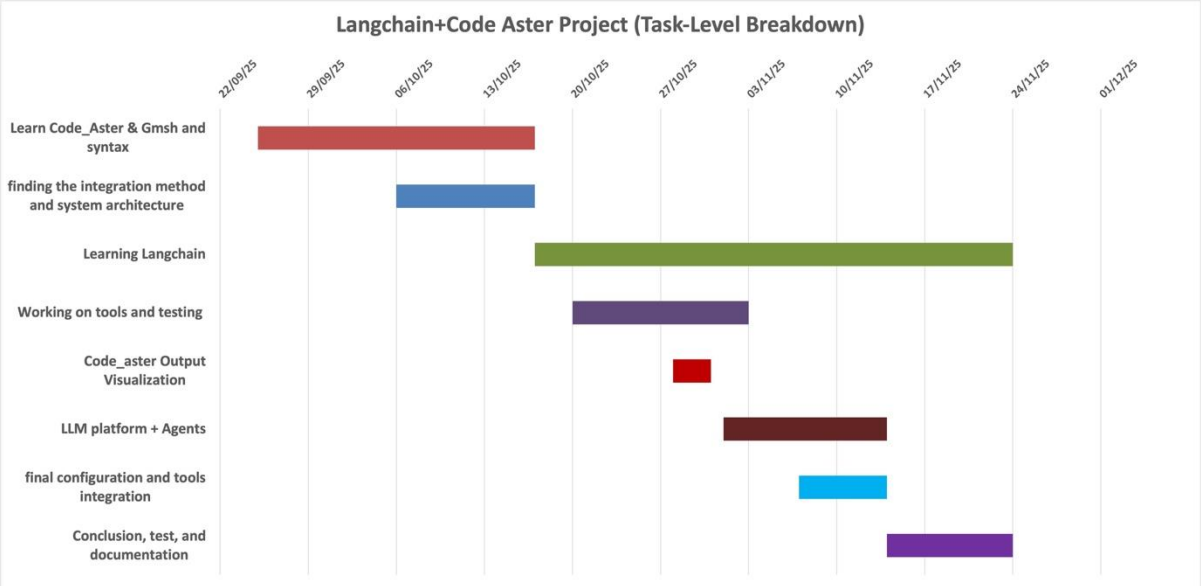


fig 01. task-level Gantt chart

2. LangChain Overview

LangChain is an open-source framework designed to facilitate the development of applications powered by Large Language Models (LLMs). Rather than using an LLM as a standalone chatbot, LangChain enables it to interact with external tools, APIs, and databases, allowing it to perform complex reasoning and take real actions. The framework provides the building blocks for integrating natural language understanding with structured programmatic control, which makes it ideal for intelligent automation systems like the one developed in this project.
At its core, LangChain organizes an application into several modular components:

- LLM (Large Language Model): the central reasoning engine that interprets natural language prompts and generates responses.
- Tools: Python functions or APIs that the model can call to perform specific tasks, such as file generation, running a simulation, or visualizing data.
- Agents: decision-making modules that determine which tool to use and when to use it based on the user's intent.
- Chains: sequences of processing steps that link multiple tools or reasoning stages together.
- Memory: components that store contextual information across interactions, allowing for persistent and adaptive behavior.

In this project, LangChain was selected because it allows seamless integration between AI reasoning and engineering computation tools. By defining each Code_Aster-related operation as a separate LangChain Tool, the LLM could autonomously manage the full workflow: generating geometry and mesh files, creating solver input decks, executing the analysis, and visualizing results. This modular and interpretable design makes it easier to debug, extend, and scale the system while maintaining transparency in every step of the process.
LangChain has already been adopted by several major organizations and open-source projects pursuing similar intelligent automation goals. For instance, OpenAI's ChatGPT plugins employ LangChain-like architectures to enable large language models to call external APIs securely; Zapier AI Actions use LangChain to connect LLMs to thousands of third-party web services; and Notion AI leverages it to perform dynamic reasoning and content generation within documents. Furthermore, open-source initiatives such as OpenDevin and AutoGPT use

LangChain to build autonomous software engineering agents capable of complex decision-making and task execution.
In the context of this internship, LangChain served as the intelligent bridge between natural-language reasoning and numerical computation, transforming traditional finite element workflows into adaptive, AI-assisted pipelines.

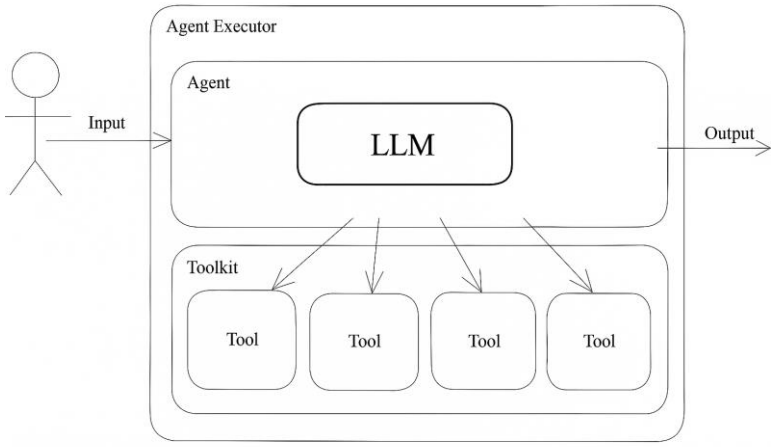


fig 02. *LangChain Agent Architecture and Interaction Flow*

3. Code_Aster Overview

Code_Aster is an open-source Finite Element Analysis (FEA) software developed and maintained by *Électricité de France (EDF)*. It is one of the most advanced and well-documented open-source solvers for structural and thermal mechanics, widely used in research and industry for simulations involving elasticity, plasticity, dynamics, contact, fracture, and thermomechanical coupling. Its high flexibility and script-based design make it particularly suitable for integration with automated and AI-driven workflows such as the one developed in this project.

3.1. General Description

Code_Aster can be operated both through a graphical user interface (GUI) — typically via *Salome-Meca*, which provides pre- and post-processing tools — or directly from the command line, where it is invoked by the execution manager `as_run`.
It also integrates smoothly with Gmsh, an open-source geometry and meshing software, allowing users to define geometric models (`.geo`), convert them into mesh files (`.med`), and import them into Code_Aster for simulation.
The solver supports both linear and nonlinear analyses, static and dynamic problems, and a wide range of element types and material behaviors. Its modular input structure enables the user to define geometry, materials, boundary conditions, loads, and output options in a single, script-like file.

3.2. Syntax and Input Files

A standard Code_Aster analysis typically requires three key input files:

- `.comm` file: defines the analysis setup using Code_Aster commands (similar to Python syntax).
- `.med` file: contains the mesh geometry, often generated by Gmsh or Salome-Meca.
- `.export` file: specifies file paths and parameters for running the solver.

The `.comm` file is the core of the analysis. This syntax allows full control of the simulation logic while maintaining transparency and reproducibility. Because Code_Aster's input language is highly structured and close to Python, it can be easily generated or modified by external scripts including those written in Python and managed by LLMs through LangChain.

3.3. Execution and API Integration

Code_Aster can be executed in multiple ways depending on the user environment and automation level:
Graphical Mode (Salome-Meca): suitable for interactive modeling, meshing, and result visualization.
Command-Line Mode: the solver is executed using the `as_run` command, which reads the `.export` file and launches the analysis. Example:

```
as_run model.export
```

Python or API-Based Automation: Code_Aster can also be integrated into Python scripts via subprocess calls or direct API bindings (e.g., `code_aster.CA`). This enables external control and automation of the solver, allowing it to be triggered dynamically by higher-level AI agents. In this project, the execution was handled programmatically using a Python wrapper (`run_aster.py`), which invoked Code_Aster via the `subprocess` module. This made it possible for the LangChain agent to autonomously prepare, run, and monitor the analysis process while interacting with the solver as an external computational tool.

3.4. Advantages over Commercial Alternatives

Compared to proprietary solvers such as Abaqus, ANSYS, or COMSOL Multiphysics, Code_Aster offers several key advantages for research and automation:
Open-source and fully scriptable: ideal for integration with AI or algorithmic systems.
Extensive documentation and validation database: over 2,500 test cases verified by EDF.
High flexibility: supports Python-like logic, loops, and conditional execution inside `.comm` files.
Cost-effective and transparent: accessible to universities and small companies without licensing constraints.
Integration-ready: compatible with Gmsh, Salome-Meca, and modern APIs, making it an excellent choice for developing intelligent, automated FEA pipelines.

4. System Architecture

The developed system follows a modular and layered architecture**,** designed to integrate an AI reasoning engine (LangChain Agent) with the open-source finite element solver Code_Aster**.** The objective of this architecture is to automate the full simulation workflow—from natural language input to numerical execution and visualization—while keeping each component independent, reusable, and transparent.
The overall workflow begins when a user provides a natural language instruction, such as *"simulate a steel beam with a height of 0.2 m under a 1.5 kN load"*. The LangChain Agent, powered by a Large Language Model (LLM), interprets this instruction, extracts the parameters, and autonomously calls the appropriate tools to generate the necessary Code_Aster input files. Once all input files are prepared, the agent executes the solver and visualizes the results.
The system architecture can be divided into the following main layers:

4.1. High-Level Workflow

1. **User Input:**
   The user communicates with the system in plain language. The prompt is passed to the LangChain Agent, which interprets the engineering problem.
2. **LLM Reasoning and Tool Selection:**
3. The LLM (via the OpenAI API) interprets the user's request and intelligently selects the appropriate next tool — such as geometry creation, mesh generation, `.comm` and `.export` file preparation, analysis execution, or result visualization.
4. **Tool Execution and File Generation:**
   Each engineering operation is encapsulated as an independent LangChain **Tool**, defined as a Python function.
   The agent calls these tools in sequence to produce:
   - `.geo` → geometry file (via **geo_generator.py**)
   - `.med` → mesh file (via **mesher_gmsh.py**)
   - `.comm` → solver input (via **comm_generator.py**)
   - `.export` → execution file (via **export_generator.py**)
5. **Simulation Execution:**
   The **run_aster_tool.py** module triggers Code_Aster (via the `as_run` command or Python subprocess), using the generated input files.
6. **Visualization and Validation:**
   The output results are post-processed using **visualize_tool.py**, and the generated data is validated through **validators.py** to ensure that file paths, syntax, and solver results are consistent.

4.2. Folder Structure and Responsibilities

The system's source code is organized into modular directories, each serving a specific function:

- **`llm/`**
  Contains the interfaces to different language models:
  - `local_llm.py` — connects to a locally hosted model (e.g., Ollama).
  - `mock_llm.py` — used for offline testing.
  - `openai_llm.py` — connects to cloud-based OpenAI models via API.
- **`tools/`**
  Hosts all LangChain Tool definitions — the building blocks of the automation pipeline:
  - `geo_generator.py` → creates geometry scripts for Gmsh.
  - `mesher_gmsh.py` → runs Gmsh to produce `.med` mesh files.
  - `comm_generator.py` → generates Code_Aster input commands (`.comm`).
  - `export_generator.py` → builds the `.export` file for solver execution.
  - `run_aster_tool.py` → runs Code_Aster and monitors the execution.
  - `visualize_tool.py` → visualizes stress/strain distributions and time histories.
  - `validators.py` → checks file integrity, parameter ranges, and format compliance.
- **`workspace/`**
  Acts as a temporary directory where all generated files and solver results are stored during each run. It is automatically cleared or updated for new analyses.
- **Root Files:**
  - `main_agent_hybrid.py` — the core script that launches the LangChain agent, defines available tools, and manages the reasoning–execution loop.

- `env_loader.py` — loads environment variables and API keys from `.env`.
- `pyproject.toml` and `requirements.txt` — define dependencies, package metadata, and reproducibility setup.
- `README.md` — provides documentation and setup instructions for the project.

4.3. Data and Execution Flow

The data flow of the system can be summarized as follows:

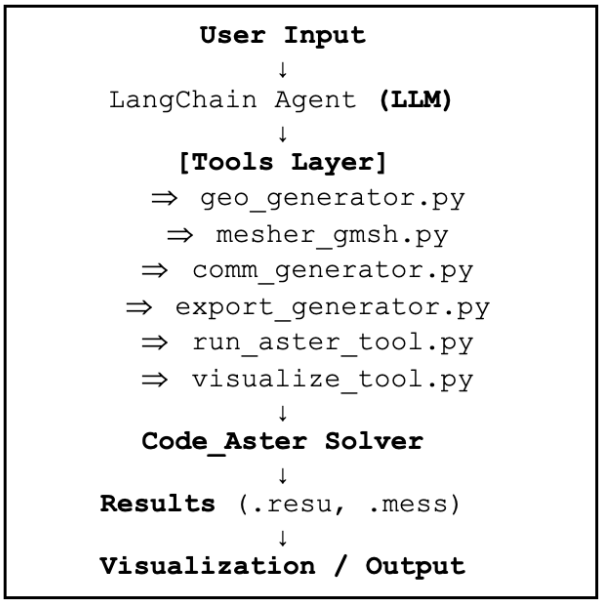


fig 03. system *Architecture and data flow*

This pipeline ensures that the system operates autonomously and transparently — each step is traceable, and every intermediate file can be inspected. Such a modular structure enables scalability: new tools can easily be integrated into the workflow without modifying the existing core.

5. Code Implementation

This section describes the main Python modules that implement the automated simulation pipeline. Each module is exposed to the LangChain Agent as a Tool, so that the LLM can invoke them when needed during the reasoning process.

5.1. `geo_generator.py` — Geometry Creation

The `geo_generator.py` module is responsible for creating the Gmsh geometry script (`.geo` file) from high-level parameters provided by the agent.
Typical inputs include geometric and material properties such as beam length, cross-section dimensions, mesh density, and group names for boundary conditions.

The tool:

- Validates the received parameters (e.g., positive dimensions, reasonable mesh size).

- Writes a `.geo` file in the `workspace/` directory, defining points, lines, surfaces/volumes, and physical groups.
- Ensures that the naming convention of physical groups (e.g. `FIXED_END`, `LOAD_EDGE`, `BEAM`) is consistent with what will later be used in the `.comm` file.

From the agent's point of view, the tool receives a structured dictionary of parameters and returns the **path to the generated `.geo` file**, which becomes the input for the meshing step.

5.2. `mesher_gmsh.py` — Meshing Process

The `mesher_gmsh.py` module takes the `.geo` file produced by `geo_generator.py` and converts it into a mesh file in MED format (`.med`), which is compatible with Code_Aster.

Its core responsibilities are:

- Calling Gmsh (via command line or Python API) to:
  - Generate the discrete mesh.
  - Apply the requested element size / number of elements.
  - Preserve physical groups as element sets.
- Saving the final mesh in `.med` format in the `workspace/` directory.
- Returning the path to the **`.med` file** for the subsequent solver configuration.

This separation of geometry and meshing into two tools keeps the workflow flexible: the agent can change the mesh density without redefining the whole geometry.

5.3. `comm_generator.py` — Solver Configuration

The `comm_generator.py` module generates the Code_Aster command file (`.comm`). It translates the problem specification (material, loads, boundary conditions, analysis type) into valid Code_Aster syntax.

The tool:

- Receives as input:
  - Path to the `.med` mesh file.
  - Mechanical and material parameters (E, ν, density, load values, load type).
  - Analysis options (static/dynamic, 2D/3D model, element type).
- Writes a `.comm` file that typically includes:
  - `LIRE_MAILLAGE` to read the mesh.
  - `DEFI_MATERIAU` and `AFFE_MATERIAU` for material assignment.
  - `AFFE_MODELE` to define the finite element formulation.
  - Definitions of boundary conditions and loads using the physical groups from Gmsh.
  - A solver call such as `MECA_STATIQUE` or `DYNA_NON_LINE`.
  - Output instructions (`IMPR_RESU`, `IMPR_TABLE`, etc.).

The output of this tool is the path to the generated `.comm` file, which fully describes the analysis in Code_Aster.

5.4. `export_generator.py` — Export Preparation

The `export_generator.py` module creates the **`.export`** file required by the `as_run` execution manager of Code_Aster. This file tells Code_Aster which input files to use and where to store the results.

Key tasks:

- Linking logical units (e.g. `20`, `80`) to real files:
  - the `.med` mesh,
  - the `.comm` script,
  - the output result files.
- Specifying the working directory inside `workspace/`.
- Defining run options such as memory, time limit, and version if needed.

The tool returns the path to the `.export` file, which is then used by the execution tool. By generating this file programmatically, the agent can run multiple analyses with different configurations without manual intervention.

5.5. `run_aster_tool.py` — Simulation Execution

The `run_aster_tool.py` module is responsible for launching Code_Aster and monitoring the simulation.

Its behavior includes:

- Receiving the path to the `.export` file from the previous tool.
- Calling Code_Aster through a subprocess (e.g. `as_run model.export`).
- Capturing the solver log (`.mess` file) and detecting basic errors (non-zero return code, missing output files).
- Returning a structured summary to the agent, including:
  - status of the run (success / failure),
  - paths to the main result files (e.g. `.resu`, `.rmed`, `.mess`).

Because this logic is encapsulated in a tool, the LangChain agent can decide to re-run the simulation with modified parameters if something goes wrong or if additional analyses are needed.

5.6. `visualize_tool.py` — Results Visualization

The `visualize_tool.py` module handles post-processing and visualization of the solver results. It focuses on turning raw numerical output into plots or data structures that can be inspected by the user.

Typical features are:

- Reading Code_Aster result files (e.g. via `rmed` or exported tables).
- Extracting relevant quantities such as displacement fields, stress profiles, or reaction forces.

- Generating plots (e.g. bending moment diagrams, deflection curves, time histories) and saving them as images in the `workspace/` directory.
- Optionally returning numerical summaries (max stresses, displacements) that the LLM can use to formulate a textual explanation of the results.

From the user's perspective, this tool closes the loop: a natural language request leads to an automated simulation, and the results are presented both graphically and in plain language by the agent.

6. Results and Discussion

This section presents the results obtained from the automated simulation pipeline and discusses the accuracy, clarity, and performance of the system. The beam example shown here was generated through the interactive "chat mode," where the user specifies the geometry, loading, material, and mesh parameters in natural language and the LangChain agent completes the entire analysis autonomously.

6.1. Example of Successful Runs

The complete set of generated files, logs, and visual outputs is provided in the Appendix.

The following section offers a detailed explanation and interpretation of the results produced. A typical interaction begins with a natural-language description of the problem:

```
User:
5m beam, 20 elements, height 0.35m, width 0.25m, load 1300 N

AI Agent:
What is the beam material?

User:
concrete c25
```

Because *concrete C25* was not in the local material database, the agent automatically initiated an online search to retrieve the mechanical properties. After confirmation, the following steps executed automatically: Geometry generation (`model.geo`), Meshing with Gmsh (MED format), Code_Aster input preparation (`model.comm`), Export creation for solver (`model.export`), Execution of Code_Aster, Post-processing & visualization.

All steps completed successfully, and the system generated:

- A complete set of solver outputs
- Six engineering plots
- A free-body diagram
- Log files for reproducibility
- Numerical summaries (max deflection, rotation, internal forces)

The workflow required **no manual intervention** after the initial user description.

6.2. Interpretation of Generated Plots

The system produces six engineering visualizations, which together provide a complete overview of the beam's structural response.

- **Free-Body Diagram:** A clean illustration is generated showing: The fixed support, The applied tip force (exp. 1.5 kN or 1300 N depending on user input), The beam length, The computed reaction forces and moment at the support This makes the visualization understandable even to readers unfamiliar with Code_Aster's output structure.
- **Vertical Deflection Curve (DY):** The deflection plot shows the transverse displacement along the beam length.
- **Rotation Curve (DRZ):** The rotation profile shows the slope of the deflected shape.
- **Bending Moment Contour (MFZ):** A color-coded contour shows the bending moment.
- **Bending Moment Diagram (MFZ):** A standard engineering diagram.
- **Shear Force Diagram (VY):** The shear diagram.

This validates both the loading and mesh partitioning. Overall, the results are fully consistent with classical beam theory, illustrating both the correctness of the computational pipeline and the reliability of automated input generation.

6.3. Performance and Automation Improvements

The current pipeline demonstrates substantial automation improvements:

- **End-to-end process automation:** The LLM handles the entire workflow without code editing by the user.
- **Error detection:** The system identifies missing materials, invalid parameters, missing physical groups, and mesh export issues.
- **Automatic online search:** If a material is unknown, the agent retrieves mechanical properties from external sources and interprets them.
- **Unified logging:** Logs include solver messages, subprocess outputs, and tool-by-tool status indicators.
- **Consistent file handling:** All generated files (.geo, .med, .comm, .export, .png plots) are stored in a structured way under `workspace/`.
- **Portability:** The entire system runs on Windows (STA DATA pipeline) and macOS (developer environment).

6.4. Challenges and How They Were Solved

1. Material Database Limitations

Challenge: Unknown materials caused pipeline failure.
Solution:

- Added web search fallback
- Implemented numeric extraction helper
- Implemented parameter validation checks

2. MED File Compatibility

Challenge: Code_Aster required MED version 4.0.0, whereas Gmsh outputs newer versions.
Solution:

- Enforced MED version conversion inside `mesher_gmsh.py`
- Added automatic detection of incompatible mesh versions

3. Directory and Path Handling on Windows

Challenge: Mixed slashes and missing executable paths caused Code_Aster to fail.
Solution:

- Normalized paths using `os.path`
- Implemented absolute paths for export file
- Added a global configuration for Code_Aster executable location

4. Visualization Scaling Issues

Challenge: Some matplotlib figures returned warnings about tight layout.
Solution:

- Adjusted figure sizes and padding
- Added auto-annotation for maxima
- Standardized all plots (colors, fonts, DPI)

5. LLM Parameter Extraction Errors

Challenge: User instructions sometimes ambiguous or partially missing.
Solution:

- Multi-step question-answer mode added ("What is the beam material?")
- Validators enforce numeric types, positive ranges, and safe defaults

7. Conclusion and Future developments

7.1. Achievements

This project demonstrates the feasibility of combining Large Language Models (LLMs) with the Code_Aster finite element solver to automate structural analysis tasks. Despite being a prototype developed in 250 hours approximately two months, the system successfully:

- Interprets natural-language engineering descriptions
- Generates geometry, mesh, solver input, and export files
- Executes Code_Aster automatically
- Produces visualizations of deflection, rotation, bending moment, and shear force
- Provides an explainable, step-by-step workflow for the entire pipeline

Although the current version focuses on a single, simple structural element (a cantilever beam), it establishes a clear foundation for integrating AI-driven reasoning with deterministic numerical simulation tools.

7.2. Key Learnings

Through the development of this prototype, several important insights were gained:

- LLMs can reliably orchestrate multi-step engineering workflows when equipped with well-designed tools.
- Code_Aster's script-based architecture makes it highly suitable for AI-driven automation.
- Robust validation layers are essential to prevent malformed inputs or unsafe parameter combinations.
- Even simple engineering tasks require careful coordination between geometry, mesh groups, solver syntax, and post-processing logic.
- A human-in-the-loop mechanism remains important, especially when materials, load cases, or assumptions are ambiguous.

These learnings highlight both the potential and the limitations of present-day LLM technology in computational mechanics.

7.3. Possible Extensions

The current system represents an early-stage prototype. To transform it into a fully capable engineering assistant, several directions for future development are recommended:

- **Support for more complex geometries:** 2D and 3D structural systems, shells, solids, and assemblies.
- **Nonlinear and advanced analyses:** plasticity, large deformation, time-dependent problems, seismic loading, dynamic response, and thermal–mechanical coupling.
- **Expanded load and boundary condition library:** distributed loads, thermal loads, imposed displacements, multi-load cases, and load combinations.
- **Advanced meshing strategies:** adaptive mesh refinement and user-defined element types.
- **Full control panel for Code_Aster settings:** solver parameters, convergence options, contact definitions, multi-step analyses.
- **Integration with optimization or surrogate models:** to enable parametric studies, sensitivity analysis, or AI-based design exploration.
- **Broader material database** and automatic retrieval of validated engineering properties.
- **A GUI or web dashboard** for users who prefer a visual interface instead of command-line interaction.

These extensions would significantly expand the system's ability to handle real engineering applications beyond the current single-beam demonstration.

7.4. Disclaimer

This software is a research prototype and is not intended for use in real engineering design, safety-critical applications, or professional structural analysis.

All simulations, visualizations, and numerical outputs generated by the system are provided *"as is"* without any guarantee of accuracy, completeness, or suitability for engineering decision-making.
The authors make no representations or warranties regarding the correctness of the results, and accept no liability for any consequences arising from the use of this code in real-world scenarios.
Users are solely responsible for independently verifying all results using certified engineering tools and qualified professional judgment.

8. Appendix

This appendix provides the complete set of outputs generated by the automated simulation pipeline, including screenshots, log excerpts, and diagnostic visualizations. These materials illustrate the full behavior of the system and serve as supporting evidence for the workflow described in Section 5.

8.1. Full Execution Log (Chat Mode)

The following transcript shows an example of a successful end-to-end run performed using the interactive chat mode of the system.The user provides a natural-language description of the beam, and the agent autonomously handles parameter extraction, material retrieval, geometry generation, meshing, solver execution, and visualization.

```
PS D:\Training_S.T.A.DATA\Project> python -m main_agent_hybrid --mode
chat

🟢 INTERACTIVE CHAT MODE
🤖 Please describe your beam and loading in one sentence.
   Example: '6m steel beam, 20 elements, height 0.2m, width 0.1m, load
1500 N'

🧑 You: 5m  beam, 20 elements, height 0.35m, width 0.25m, load 1300
N          # user forgot to write the material
🤖 What is the beam material?
🧑 You: concrete c25

⚠️ Material 'concrete c25' is not in the local database.
❓ Enter material properties manually or search online? (manual/search):
search

🌐 Searching online…

🔎 Search result summary:
 Here are the standard mechanical properties for **concrete C25** (also
known as C25/30):

- **Young's modulus (E):**
  \( E = 30 \) GPa = **30 × 10⁹ Pa**

- **Poisson's ratio (ν):**
  \( \nu = 0.2 \) (dimensionless)

- **Density (ρ):**
  \( \rho = 2400 \) kg/m³

**References:**
- [EN 1992-1-1:2004 (Eurocode
2)](https://eurocodes.jrc.ec.europa.eu/showpage.php?id=138)
- [The Constructor: Properties of
Concrete](https://theconstructor.org/concrete/properties-of-
concrete/522/)
```

```
**Summary Table:**

| Property          | Value           |
|-------------------|-----------------|
| Young's modulus   | 30 × 10⁹ Pa     |
| Poisson's ratio   | 0.2             |
| Density           | 2400 kg/m³      |

Please extract numeric values from the above result:
Enter E (Pa): 30000000000
Enter NU: 0.2
Enter RHO (kg/m³): 2400

💠 Final parameters (raw):  {'length_m': 5, 'n_elems': 20, 'HY': 0.35,
'HZ': 0.25
, 'FY': 1300, 'material': 'concrete c25', 'E': 30000000000.0, 'NU':
0.2, 'RHO':
2400.0}
📄 Final parameters going into pipeline: {'length_m': 5.0, 'n_elems':
20, 'E': 210000000000.0, 'NU': 0.3, 'RHO': 7850.0, 'HY': 0.35, 'HZ':
0.25, 'FY': 1300.0}

⚙️ Starting simulation pipeline...


🚀 Running: generate_geo
✅ Generated model.geo
Length = 5.0 m, Elements = 20, Nodes = 21
Groups defined: BASE, TIP, BEAM
→ Saved to D:\Training_S.T.A.DATA\Project\workspace\model.geo
✅ Step 'generate_geo' OK.


🚀 Running: mesh_with_gmsh
✅ Meshing completed successfully.
MED file saved at: D:\Training_S.T.A.DATA\Project\workspace\model.med
🔧 MED version forced to 4.0.0 for Code_Aster compatibility.

--- Gmsh Output (first 500 chars) ---
...
✅ Step 'mesh_with_gmsh' OK.


🚀 Running: generate_comm
✅ Generated model.comm successfully.
✅ Step 'generate_comm' OK.


🚀 Running: generate_export
✅ Export file generated: D:\Training_S.T.A.DATA\Project\workspace\
model.export
✅ Step 'generate_export' OK.
```

```
🚀 Running: run_code_aster
🔧 Running Code_Aster: C:\\STADATA\\CodeAster19\\v2019\\bin\\as_run.bat
D:\Training_S.T.A.DATA\Project\workspace\model.export
✅ Code_Aster run completed successfully.
Export: D:\Training_S.T.A.DATA\Project\Version
0.2.2\workspace\model.export

--- First lines of output ---


D:\Training_S.T.A.DATA\Project\workspace>echo OFF
Active code page: 65001


------------------------------------------------------------------------
---------
 Code_Aster execution


<INFO> prepare environment in C:\Users\EX-GIGI\AppData\Local\Temp\EX-
GIGI-DESKTOP-MVF74MH-interactif_1016


------------------------------------------------------------------------
---------
 Copying datas


copying
...\workspace\model.comm...                                    copying

...\workspace\m...
✅ Step 'run_code_aster' OK.


📊 Visualization phase ...

🚀 Running: visualize_resu
D:\Training_S.T.A.DATA\Project\tools\visualize_tool.py:84:
  plt.tight_layout()
D:\Training_S.T.A.DATA\Project\tools\visualize_tool.py:107:
  plt.tight_layout()
✅ Visualizations saved:
📉 Deflection: workspace\beam_job_def.png
📈 Rotation: workspace\beam_job_rot.png
📊 Nodes: 21 | Max DY = -0.000 mm | Max DRZ = -0.0000 rad
✅ Step 'visualize_resu' OK.


🚀 Running: visualize_stress
✅ Internal force plots generated:
```

```
🔘 Shear: workspace\shear_diagram.png
🔘 Moment: workspace\moment_diagram.png
📊 Nodes: 21 | VY range: 1300.0 → 1300.0 N | MFZ range: 0.0 → 6500.0
N·mm
✅ Step 'visualize_stress' OK.


🚀 Running: visualize_mfz_contour
✅ MFZ contour saved: workspace\mfz_contour.png
📊 MFZ range: 0.00 → 6.50 kN·m
📏 Beam length: 5.00 m | Nodes: 21
✅ Step 'visualize_mfz_contour' OK.


🚀 Running: visualize_freebody
✅ Free-body diagram generated.
🖼 Image: workspace\freebody.png
📏 Beam length: 5.00 m
📍 Support: fixed
🔁 Reactions at N1: Fx=0.00 N, Fy=-1300.00 N, Mz=-6500.00 Nmm
🎯 Loads: 1 found from model.comm
✅ Step 'visualize_freebody' OK.


🎯 All steps completed successfully — Simulation finished!
```

This transcript demonstrates that the pipeline requires **no manual intervention** after the initial user query.

The system produces **six engineering plots** plus a **free-body diagram**, all stored in the `workspace/` directory. Each figure corresponds to one of the analysis stages described in Section 5.

8.2. Example Generated Files

The system automatically creates the following files inside the `workspace/` directory:

- `model.geo` — geometry definition for Gmsh
- `model.med` — finite element mesh
- `model.comm` — Code_Aster solver instructions
- `model.export` — execution instructions for `as_run`
- `model.mess` — solver log file
- Images (`.png`) for all engineering plots

These files collectively represent a complete, reproducible numerical simulation.